

Local Multimodel Assistant for Visual Studio Code

1 Introduction

1.1 Context

The Multimodel AI Assistant project aims to develop an extension for the Visual Studio Code environment that integrates advanced intelligent assistance functionalities into the software development process. The main objective is to create a tool capable of supporting developers in activities such as code analysis and review, security vulnerability detection, project knowledge management, and workflow optimization through the orchestration of multiple artificial intelligence models.

The proposed system employs a multimodal architecture, in which several AI models, both local and cloud-based, collaborate to generate contextual and efficient responses. This approach enables dynamic adaptation of computational resources, performance optimization, and data confidentiality, depending on the specific characteristics of the analyzed project.

1.2 Motivation

In the current context of software development, the complexity of information systems has increased significantly, generating a growing need for tools that facilitate the automation of analysis and verification processes. Modern software projects involve thousands of lines of code, geographically distributed teams, and strict quality and security requirements. Under these circumstances, traditional manual code review methods become inefficient and error-prone to humans.

The motivation for employing an intelligent algorithm within this project arises from the necessity of optimizing the development process through the following aspects:

- *Operational efficiency:* AI systems can automatically analyze extensive code segments, rapidly identifying errors, inconsistencies, or potential vulnerabilities.
- *Continuous and adaptive learning:* The system can accumulate knowledge from previous projects, adapting to the team’s coding style and the specific domain of the application.
- *Enhanced software security:* Automated analysis modules can identify vulnerabilities that are difficult to detect through conventional methods, thus reducing security risks.
- *Data confidentiality:* Through support for local models, code analysis can be performed internally without exposing sensitive information to external services.

Therefore, the implementation of a solution based on intelligent algorithms represents an essential step toward modernizing the software development process, ensuring a balance between performance, security, and adaptability.

2 Contemporary Holdbacks

Artificial intelligence has emerged as a promising approach for automating parts of the software development process. Large language models (LLMs) and machine learning systems can analyze source code, generate documentation, and provide intelligent suggestions. However, current AI-powered tools often suffer from three critical limitations:

- *Single-model dependency:* Most tools rely on one large, centralized model, which limits flexibility and adaptability across diverse tasks.
- *Privacy and security concerns:* Cloud-based AI services require sending source code to external servers, raising confidentiality and compliance issues for organizations handling sensitive or proprietary data.
- *Lack of contextual learning:* Existing systems typically provide isolated, one-time analyses without learning from past interactions or adapting to the specific coding practices of a development team.

The scientific contribution lies in developing a framework that merges AI orchestration, knowledge-based adaptation, and privacy-aware computation into a unified assistant. This framework represents a new step toward intelligent software development environments, where artificial intelligence operates not as a single tool but as a coordinated ecosystem of models that understand, learn, and evolve with the developer’s workflow.

3 Related Work

The development of intelligent software assistants has received increasing attention in recent years, driven by advances in natural language processing, machine learning, and software engineering tools. Several approaches have been proposed to support developers in tasks such as code review, vulnerability detection, documentation generation, and automated refactoring.

3.1 OpenAI Codex

- *Data:* The original OpenAI Codex model was trained on approximately 159 GB of Python code from over 54 million public GitHub repositories, covering multiple programming languages including JavaScript, Go, Ruby, and C++. The dataset included a range of software projects from simple scripts to complex applications, curated to ensure high quality and adherence to good programming practices. Reinforcement learning from human feedback (RLHF) was incorporated in later versions to improve autonomous coding capabilities and understand real-world coding scenarios.
- *Algorithm:* Codex is based on the GPT-3 architecture fine-tuned specifically for programming tasks. It operates by analyzing input prompts—such as natural language descriptions or partial code snippets—and predicting the most probable continuation of code tokens. The model uses statistical patterns learned from the training data to generate code but does not execute or logically reason about the code. Iterative prompt refinement helps improve the quality of generated code.
- *Outcomes:* Codex can complete approximately 37% of coding requests accurately and generates working solutions for around 70% of test prompts when sampled multiple times. It supports over a dozen programming languages with highest performance in Python. Codex accelerates programming by generating code snippets, automating tasks like opening pull requests, refactoring files, and writing tests. Its limitations include occasional errors and dependence on prompt quality, making it a tool to assist rather than replace human developers.

3.2 Rethinking Code Review Workflows with LLM Assistance

- *Data:* The study used empirical data from an industrial field study conducted at WirelessCar Sweden AB, comprising semi-structured interviews and real-world experiments with two prototype LLM-assisted code review tools. The tools integrated

retrieval-augmented generation (RAG) to provide contextual information during code reviews, addressing challenges like frequent context switching and insufficient context.

- *Algorithm:* Two variations of LLM-assisted review tools were developed: one providing upfront LLM-generated reviews (AI-led co-reviewer) and another offering on-demand interaction with the LLM (interactive assistant). Both leveraged semantic search pipelines to retrieve relevant information and augment reviews with context for increased accuracy and developer support.
- *Outcomes:* The field experiment demonstrated that AI-led, upfront reviews were generally preferred by developers, although acceptance depended on the reviewer’s familiarity with codebases and pull request severity. Key benefits included reduced context switching and improved review efficiency, though concerns about false positives and trust in the AI were noted. The approach showed promise for enhanced developer experience and workflow optimization.

3.3 Enhanced Code Generation via Multi-Stage Performance-Guided LLM Orchestration

- *Data:* This work conducted a comprehensive empirical study analyzing the performance of 17 state-of-the-art LLMs on the HumanEval-X benchmark across five programming languages (Python, Java, C++, Go, Rust). The dataset included various algorithmic domains and development stages, evaluated with functional correctness and runtime metrics such as execution time and memory usage.
- *Algorithm:* The authors introduced PerfOrch, a multi-stage LLM orchestration framework that dynamically routes code generation tasks to the most suitable LLMs following a generate-fix-refine workflow. It incorporates stage-wise validation and rollback mechanisms without requiring model fine-tuning. The system uses performance guidance based on empirical profiling to select LLMs for each task context.
- *Outcomes:* PerfOrch outperformed single-model baselines with correctness rates of 96.22% on HumanEval-X and 91.37% on EffiBench-X, surpassing GPT-4o’s 78.66% and 49.11%. Additionally, it improved execution speeds for over half of the tested problems with median speedups of 17.67% to 27.66%, demonstrating both correctness and efficiency improvements. The framework’s scalability and plug-and-play design allow continual integration of new models.

3.4 Enhancing State Awareness for Multi-Step LLM Agent Tasks

- *Data:* The proposed Task Memory Engine (TME) was evaluated through case studies and comparative experiments on multi-step agent tasks, with datasets involving hierarchical task structures and sub-task relationships to test multi-step reasoning and execution consistency.
- *Algorithm:* TME is a lightweight, structured memory module that employs a hierarchical Task Memory Tree (TMT). Each node corresponds to a task step and stores input, output, status, and sub-task relationships. A prompt synthesis method dynamically generates LLM prompts based on the active node path to maintain contextual grounding and improve task execution coherence over multiple steps.
- *Outcomes:* TME led to better task completion accuracy, more interpretable agent behavior, and reduced hallucinations in multi-step LLM applications with minimal implementation overhead. The structured memory improved long-range coherence and robustness of autonomous LLM agents in executing complex, multi-step tasks.

This summary provides an overview of the data sources, core algorithms, and key results from these recent works on intelligent software assistants and LLM-based programming support systems.